

Von „KI schreibt Code“ zu „KI-Entwicklungsteam“

Warum strukturierte KI-Architektur bessere Ergebnisse liefert als ein Chat-Fenster

Das Missverständnis

Die meisten Entwickler nutzen KI so: Chat öffnen, beschreiben was sie brauchen, Code zurückbekommen. Das Ergebnis ist oft mittelmäßig — es läuft, aber die Codequalität ist fragwürdig. Die Schlussfolgerung: „KI kann nicht gut coden.“

Das ist, als würde man einem neuen Mitarbeiter am ersten Tag sagen: „Bau mir ein Feature“ — ohne Architektur-Vorgaben, ohne Code-Review, ohne Linting, ohne Tests, ohne Wissen über die bestehende Codebase. Natürlich ist das Ergebnis enttäuschend.

Die Frage ist nicht „Kann KI coden?“ — sondern „Wie manage ich KI so, wie ich Entwickler managen würde?“

Was heute möglich ist: Agent-Architektur

Claude Code unterstützt seit 2025 ein System aus **Agents** und **Skills**, mit dem sich vollständige Entwicklungs-Pipelines aufbauen lassen:

BAUSTEIN	FUNKTION	ANALOGIE
Main Agent	Orchestriert, delegiert, reviewt	Tech Lead
Sub-Agents	Spezialisierte Entwickler mit eigenem Kontext	Backend-Dev, Frontend-Dev
Skills	Wiederholbare Workflows (Lint, Test, Build)	CI/CD-Pipeline
Plugins	Bündelt Skills, Agents, Hooks — ein Install fürs Team	Toolchain-Installer

CLAUDE.md	Projektkontext: Architektur, Konventionen	Onboarding-Dokument
System Prompt	Identität und Denkweise eines Agents	Stellenbeschreibung

Der entscheidende Unterschied: Statt einem KI-Aufruf ohne Kontext entsteht ein **strukturiertes Team mit Qualitätssicherung auf jeder Stufe**.

Die Pipeline auf einen Blick

Am Beispiel eines produktiven Projekts (Cross-Platform App, Rust/React/Kotlin, ~15.000 LOC)

1	Planung Feature wird in Tasks zerlegt, Agents zugeordnet, Abhängigkeiten identifiziert
2	Implementierung + Review Spezialisierter Agent implementiert. Tech Lead reviewt auf Architektur-Konformität. Max. 2 Korrektur-Runden.
3	Qualitätssicherung Formatter → Linter → Build → Tests. Automatisch, bei jedem Feature.
4	Integration Conventional Commits, nur relevante Dateien, Architektur-Entscheidungen dokumentiert
5	Release Ein Befehl: Version Bump → Multi-Platform Build → GitHub Release

Jede Stufe hat eigene Quality Gates. Code erreicht den Commit erst nach Linting, Build und Tests — genau wie in einem menschlichen Team mit CI/CD.

Teamtauglich und messbar

Plugins lösen das „Works on my machine“-Problem: Eine komplette Pipeline — Skills, Agents, Hooks, Konfiguration — wird als Plugin paketierr und mit einem Befehl ans Team verteilt. 9.000+ Extensions existieren bereits im Ökosystem.

Skill Creator bringt Software-Engineering-Methoden auf die Meta-Ebene: Die Arbeitsanweisungen an die KI (Skills) werden selbst mit Evals, Benchmarks und A/B-

Tests geprüft und optimiert. Nicht „hoffen, dass es funktioniert“ — sondern messen, vergleichen, verbessern.

Der Mensch wird nicht ersetzt — er wird vom Programmierer zum Architekten.

Die Qualität des KI-Outputs ist eine Funktion der Architektur, nicht der KI selbst.

Basierend auf einem produktiven Projekt mit echten Releases und Nutzern. Gesamte Entwicklung über Agent-Architektur mit Claude Code.